Scenario 1: Bad Container Image Deployment

Objective

Trace an incomplete Kubernetes rollout from its first symptom to the Deployment image and Amazon ECR, then recover safely.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/bad-deploy.sh banking banking-backend banking-backend
```

The script changes the live Deployment image to a nonexistent tag. Give Kubernetes a minute to observe the failed rollout before troubleshooting. Do not run another scenario until this one has been recovered.

Situation

The banking application still serves users, but a new release has not completed.

Symptoms

- The rollout does not complete.
- New pods do not start while existing pods may remain healthy.

What You Know

- The issue began immediately after a banking backend release.
- The `banking-backend` Deployment has new pods that are not Ready.
- Existing pods may still serve traffic.

Start With the Symptom

1. Watch the rollout. It never finishes:

```
kubectl rollout status deployment/banking-backend -n banking --timeout=30s
```

1. List recent warning events in the namespace and look for `ErrImagePull`, `ImagePullBackOff`, `Failed to pull image`, or `does-not-exist`:

```
kubectl get events -n banking --field-selector type=Warning --sort-by=.lastTimestamp
```

Unavailable replicas can remain `0` in this scenario because the two old pods continue serving while Kubernetes creates one bad surge pod. Use the desired-versus-updated gap plus the image-pull event as the primary evidence.

Record the first event timestamp, failed pod, and requested image before digging deeper.

Troubleshooting

1. Check the Deployment summary.

```
kubectl get deployment banking-backend -n banking
```

Expected output: `READY` remains `2/2`, `AVAILABLE` remains `2`, but `UP-TO-DATE` is only `1`. This means the old replicas are healthy while the new rollout is incomplete.

1. Find the failed new pod.

```
kubectl get pods -n banking -l app=banking-backend
```

Expected output: two older pods are `Running`, while the newest pod is `ErrImagePull` or `ImagePullBackOff` and shows `0/1` Ready.

1. Inspect the failed pod and its recent events.

```
kubectl describe pod <pod-name> -n banking
```

Expected output: the **Events** section contains `Failed to pull image`, the requested tag ends in `does-not-exist`, and the registry reports that the image was not found.

1. Confirm the image configured on the Deployment.

```
kubectl get deployment banking-backend -n banking \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

Expected output: an ECR image URL ending in `:does-not-exist`.

1. Compare the requested tag with the images that actually exist in ECR.

```
aws ecr describe-images --repository-name dev-cluster/banking-backend \
  --region us-east-1 --query 'imageDetails[].imageTags' --output table
```

Expected output: valid image tags are listed, but `does-not-exist` is absent. The missing ECR tag is the root cause.

Important Clues

Connect the replica mismatch with `ErrImagePull` or `ImagePullBackOff`, Kubernetes events, and the ECR image list.

Root Cause

Record your conclusion from the Kubernetes and ECR evidence before you apply the fix.

Fix

Roll back to the last working revision.

```
kubectl rollout history deployment/banking-backend -n banking
kubectl rollout undo deployment/banking-backend -n banking
```

Validation

```
kubectl rollout status deployment/banking-backend -n banking
kubectl get pods -n banking
```

Confirm the application still works and every replica is up to date and Ready.

DevOps Lesson

Validate an ECR tag before changing a Kubernetes Deployment. A healthy old ReplicaSet does not mean a release succeeded.

STAR Method

Situation

Summarize the failed rollout without naming the cause first.

Task

Explain the safe recovery objective.

Action

Describe the evidence path from the rollout status to Kubernetes events and ECR.

Result

State how the rollout recovery was verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

One of our EKS releases referenced image tag `v2.4.1`, but only `v2.4.0` existed in ECR. My task was to restore a safe rollout without interrupting the old healthy replicas. I used the rollout status and Kubernetes events to confirm `ImagePullBackOff`, verified that the tag was missing in ECR, and rolled back the Deployment. The application remained available, the rollout recovered, and we added image-tag validation to the pipeline.